RAJALAKSHMI
ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23432 – Software Construction

(REGULATION 2023)

RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai-602015

Name: Pooja S

Register No: 240701386

Year / Branch / Section: II / CSE

Semester: IV

Academic Year: 2025 - 2026

INDEX

S.No.	Date	Title
1.	07-02-2026	Azure DevOps Environment Setup
2.	14-02-2026	Azure Project Setup & User Stories
3.	21-02-2026	Epics, Features, User Stories Creation
4.	28-02-2026	Sprint Planning using Azure Boards
5.	07-03-2026	Poker Estimation using Azure Boards.
6.	14-03-2026	Designing Class and Sequence Diagrams for Product Category Management System.
7.	04-04-2026	Designing Architectural and ER Diagrams for Product Category Management System.
8.	11-04-2026	Testing – Test Suites and Test Cases in Azure.
9.	18-04-2026	Load Testing and CI/CD Pipeline Setup.
10.	25-04-2026	GitHub: Project Structure & Naming Conventions.

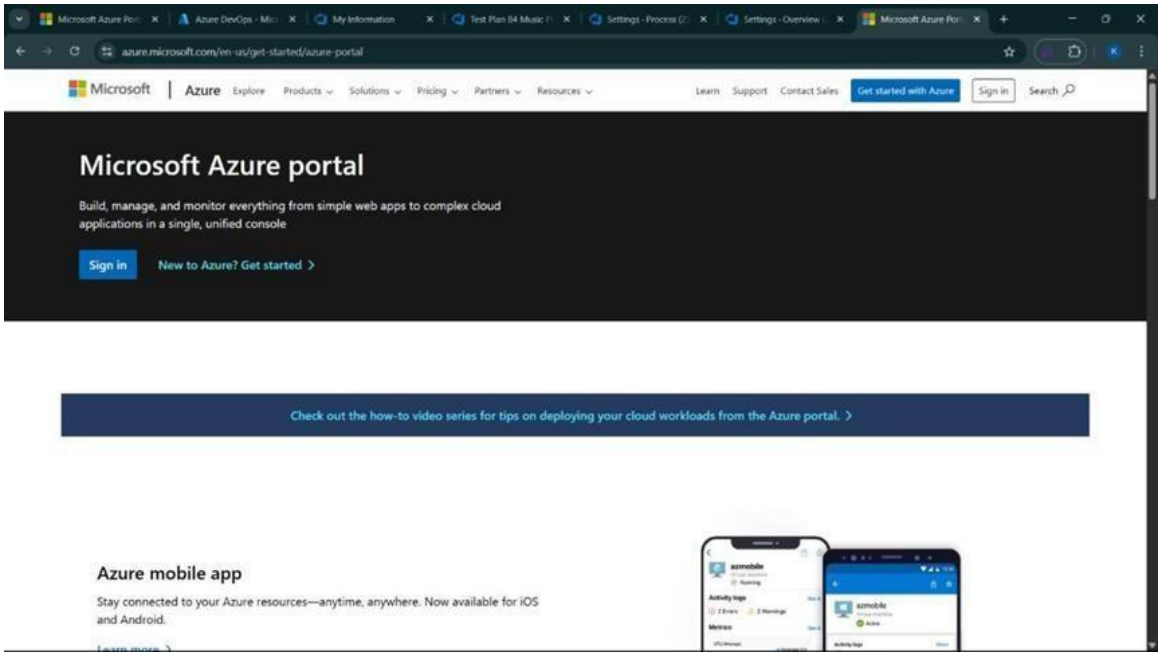
EXP NO: 01	AZURE DEVOPS ENVIRONMENT SETUP

Aim:

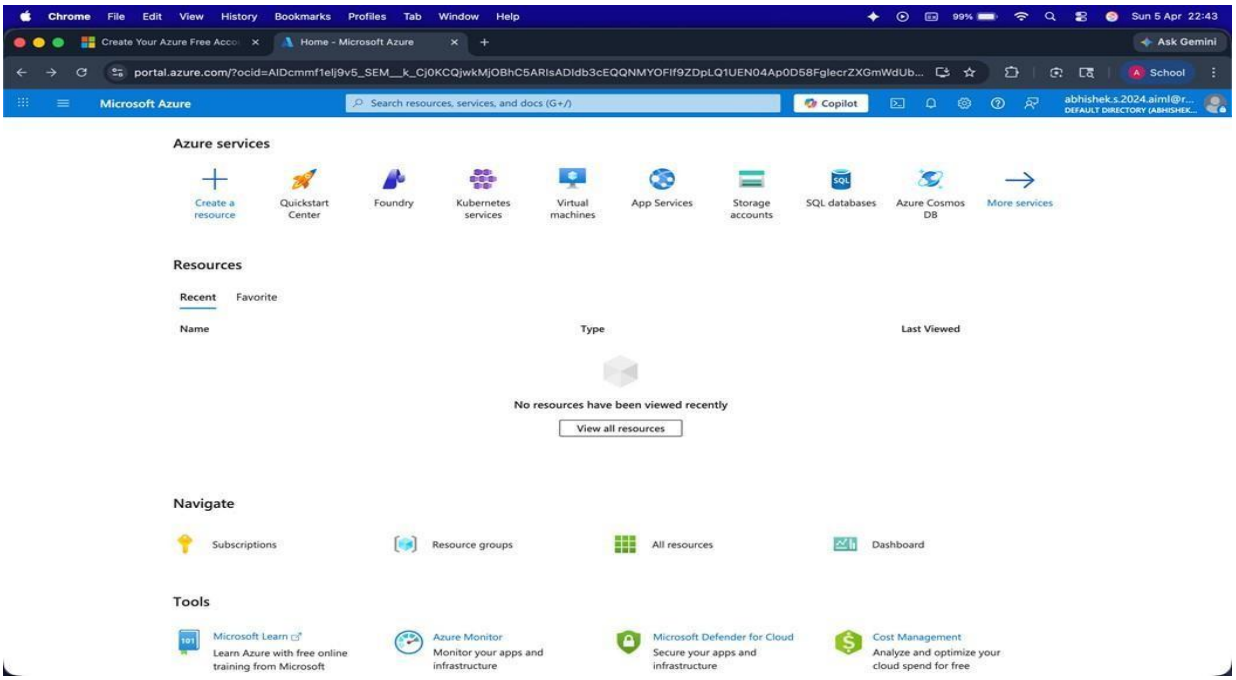
To set up and access the Azure Devops environment by creating an account and workspace for project management.

INSTALLATION

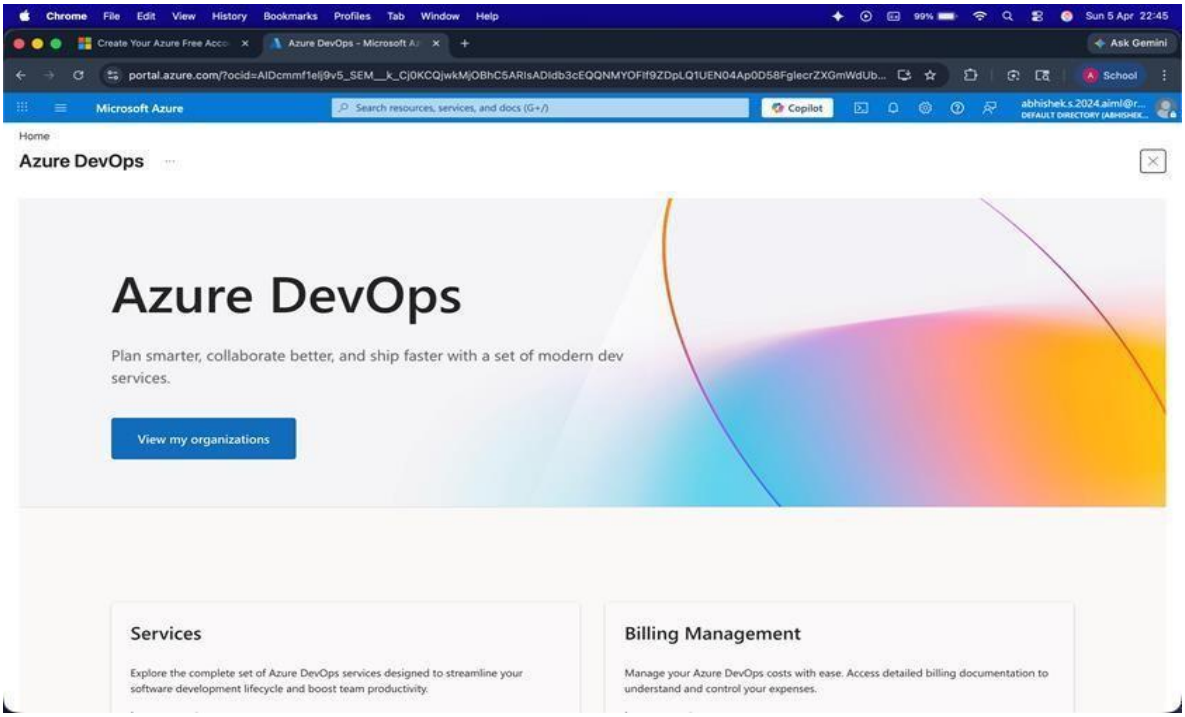
1. Open your web browser and go to: https://dev.azure.com
   Sign in using your Microsoft account credentials.
   If you do not have an account, create one using Sign Up option.



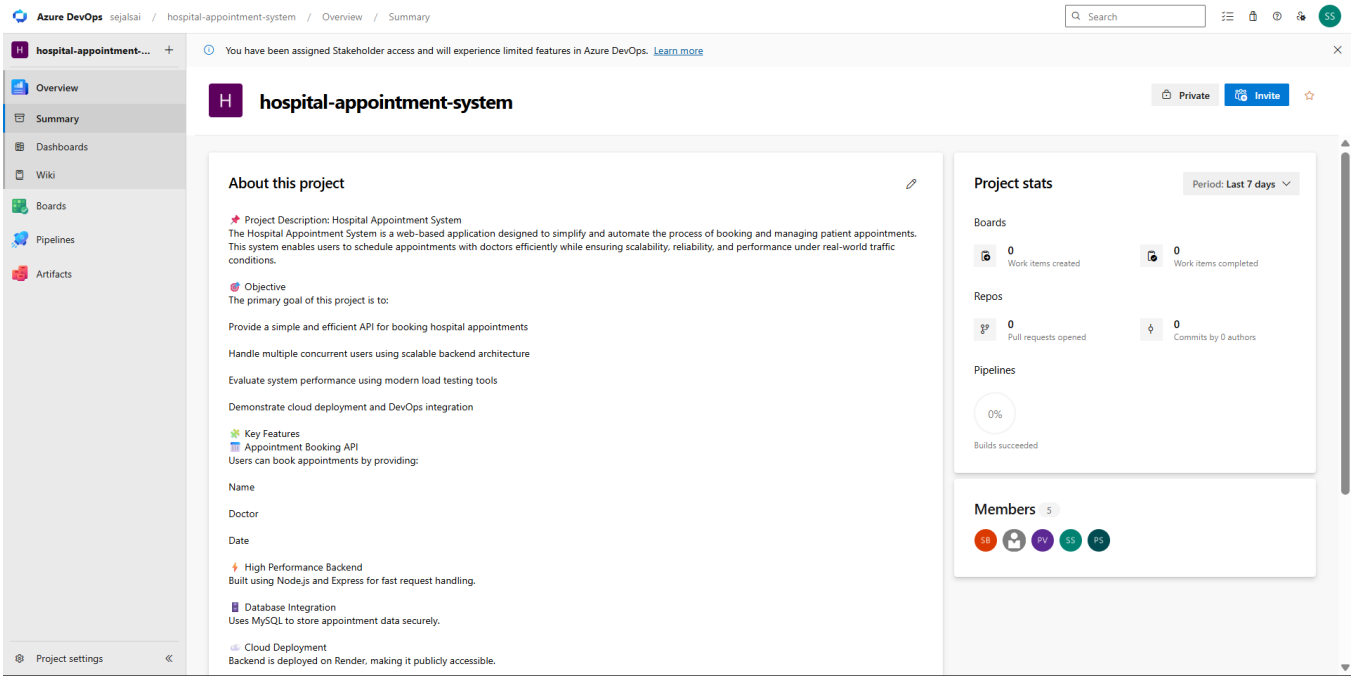
2. Azure Devops home page



3. Open DevOps environment in the Azure platform by typing ***Azure DevOps Organizations*** in the search bar.



4. Click on **New Project**, create a new organization/project, and access the Azure DevOps dashboard successfully..



Result:

Successfully accessed the Azure environment and created a new workspace/site for project management.

EXP NO: 02	AZURE PROJECT SETUP AND USER STORY MANAGEMENT

Aim:

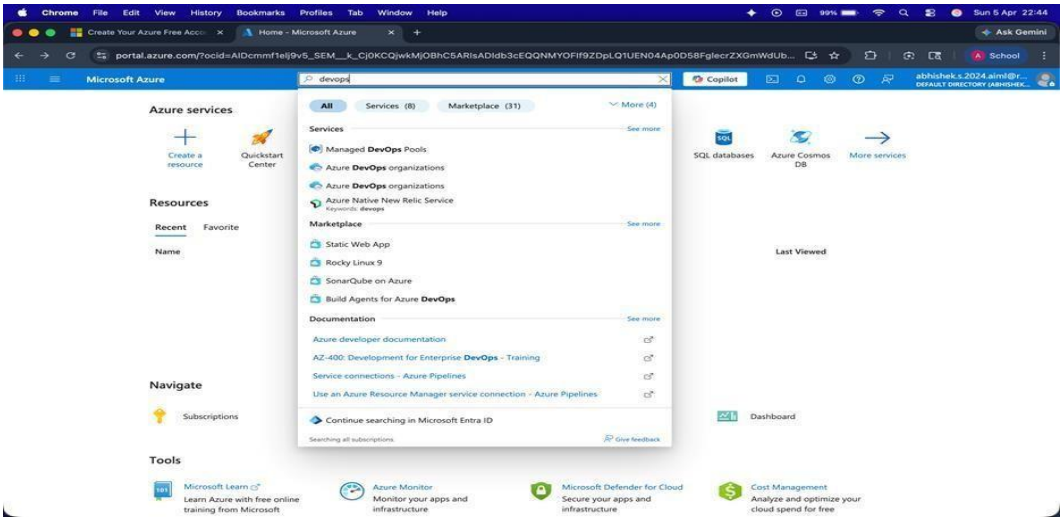
To set up an Azure DevOps project for efficient collaboration and agile work management.

- Create an Azure DevOps Account
- Create the First Project in Your Workspace

a. After setup, create your first project.
b. Click **New Project**
c. Enter details:
Name: Product Category Management System
Description: Category handling system
Template: Scrum
Access: Private/Open
d. Click Create

- Open project dashboard
- Project dashboard
- To manage user stories:

a. Go to **Boards → Work Items**
b. Click **New Work Item → User Story**
c. Enter details



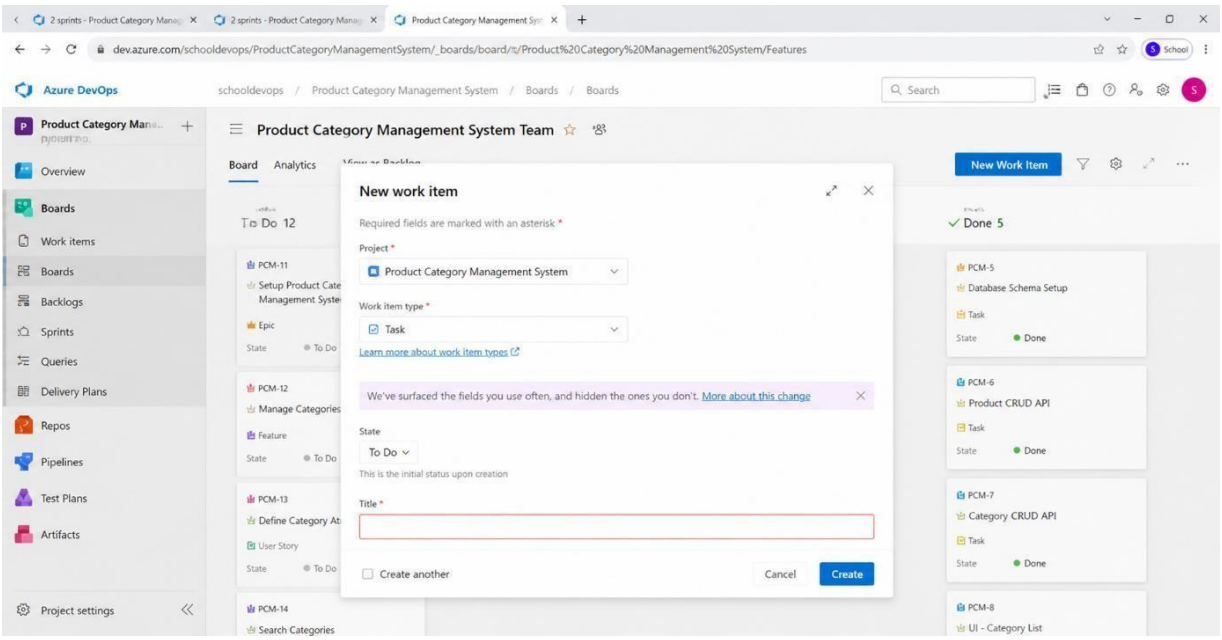
Result: Successfully created an Azure DevOps project with user story management

EXP NO: 03	SETTING UP EPICS, TASKS, AND USER STORIES FOR PROJECT PLANNING

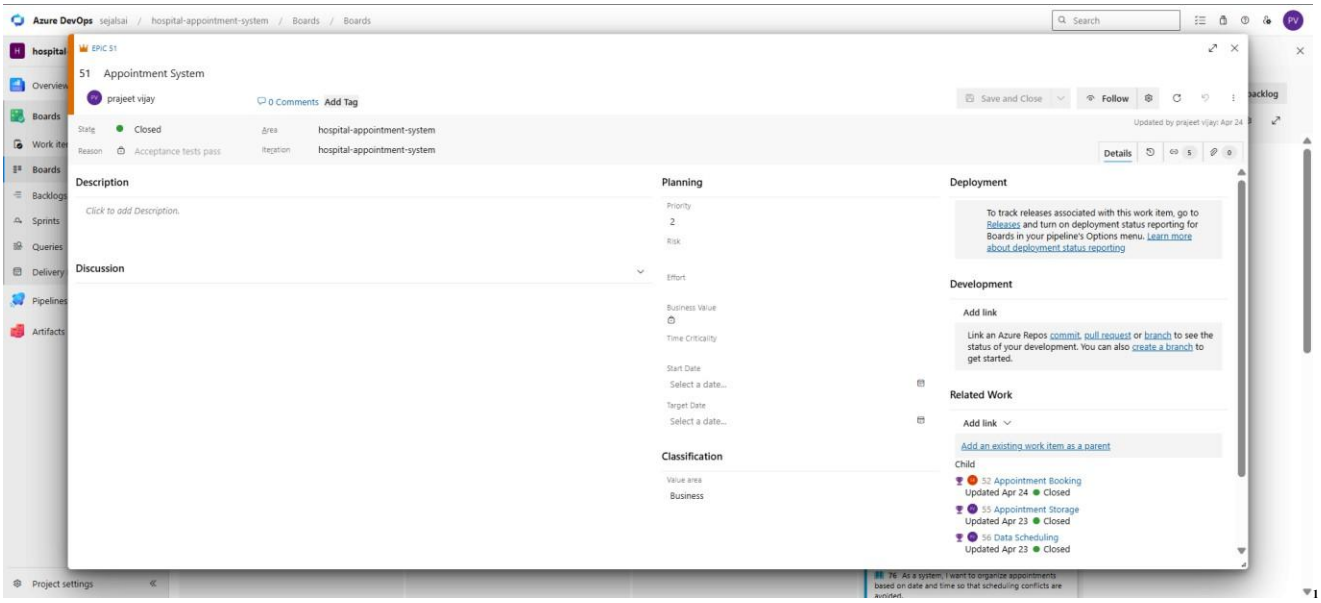
Aim:

To learn about how to create epics, tasks, user stories, and backlogs for your assigned project.

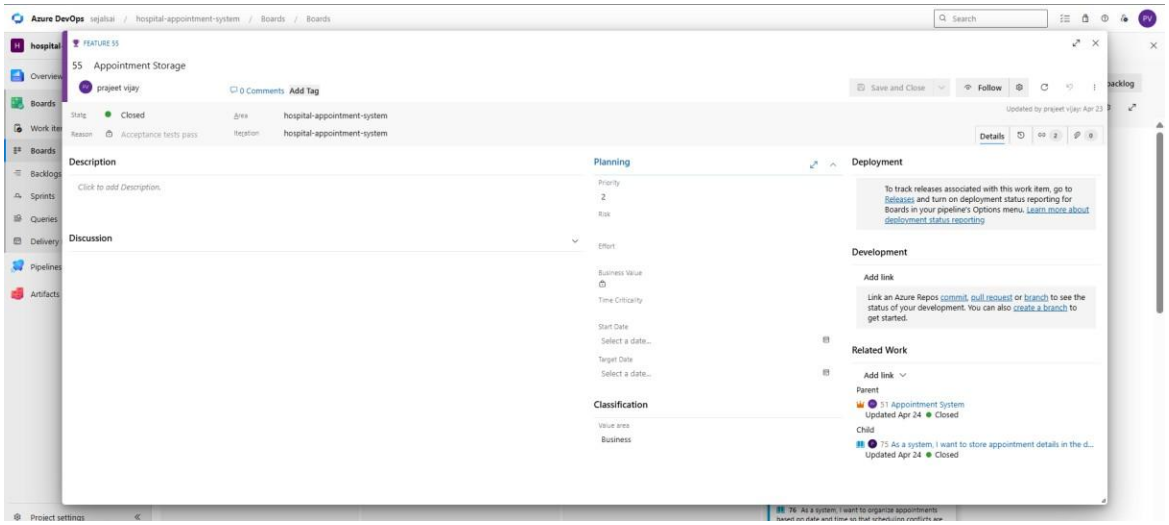
Create Epic, User Stories, Task



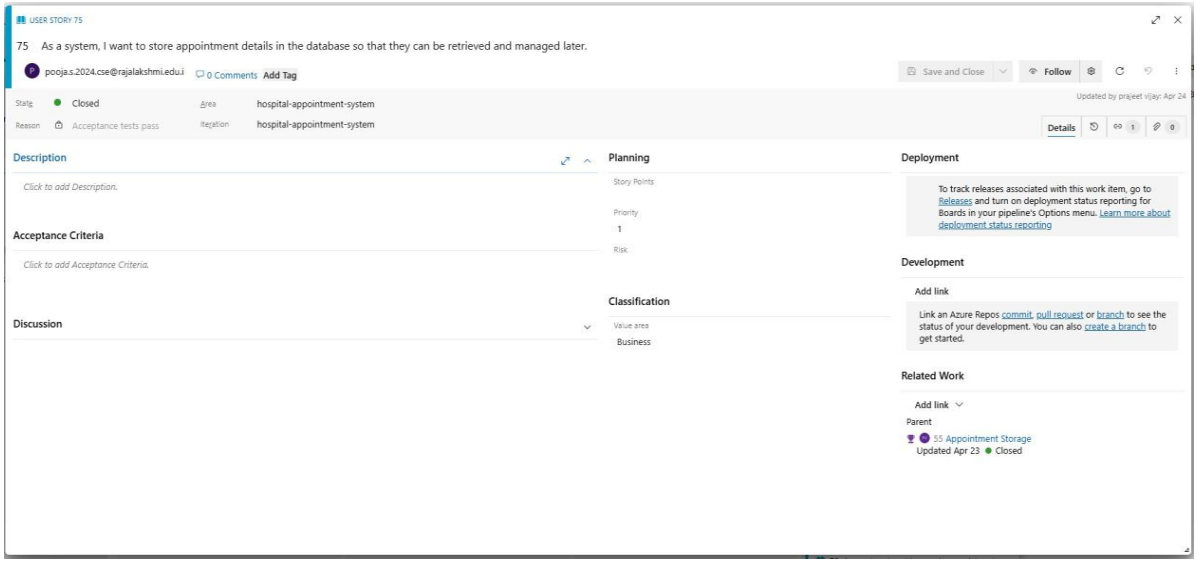
1. Fill in Epics



2. Fill in Features



3. Fill in User Story Details



Result:

Thus, the creation of epics, features, user story and task has been created succesS

EXP NO: 04	SPRINT PLANNING IN AZURE

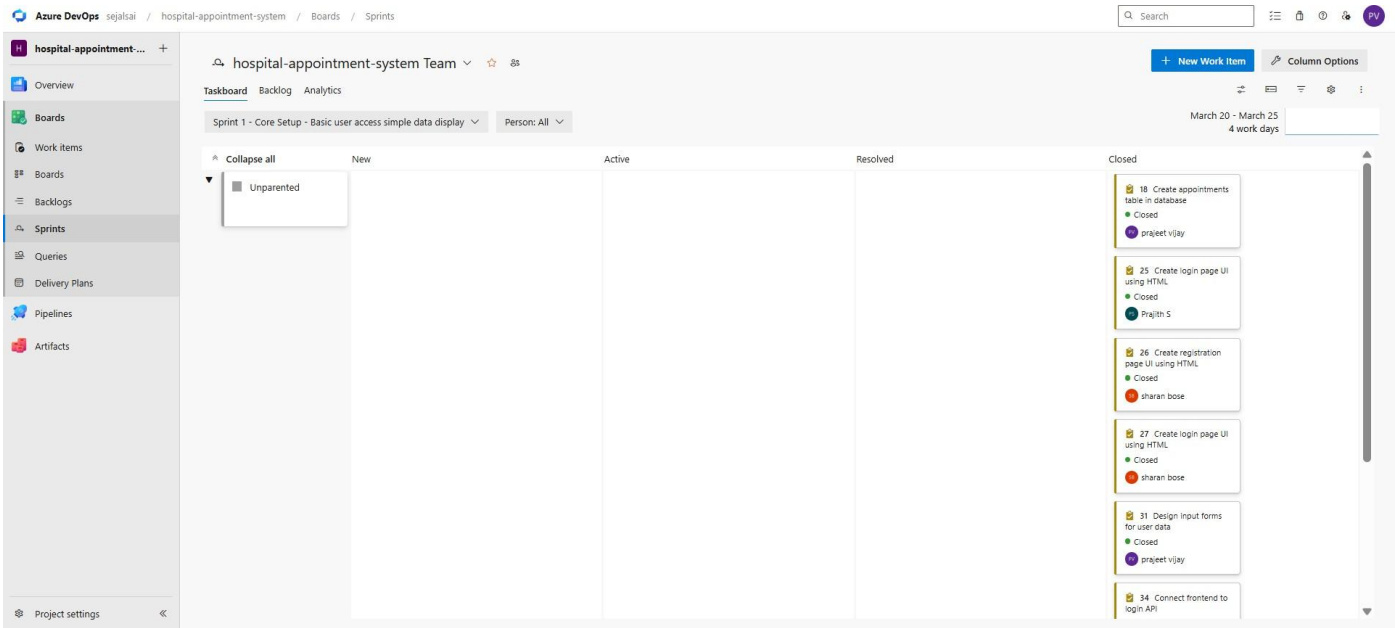
Aim:

To assign user story to specific sprint for the Hospital Appointment Management System

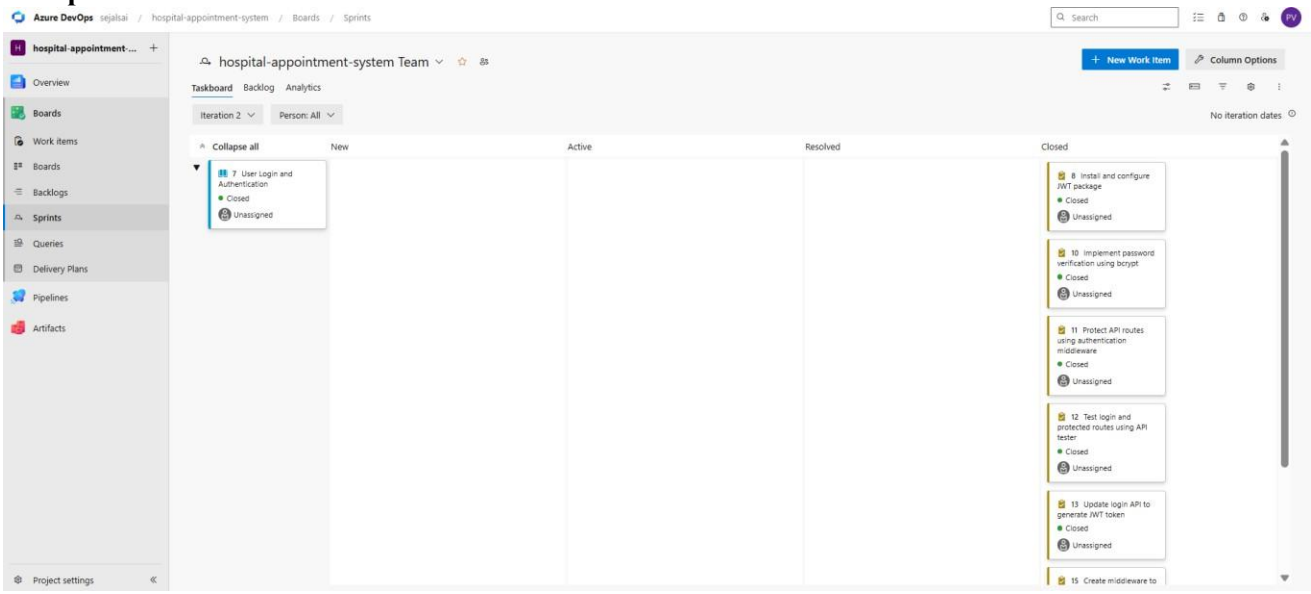
Sprint Planning

1. Open **Boards → Sprints**
2. Create:
   - Sprint 1
   - Sprint 2
3. Drag user stories into sprint

Sprint 1



Sprint 2



Result:

The Sprints are created for the Hospital Appointment Management System

EXP NO: 05	POKER ESTIMATION USING AZURE

Aim:

Create Poker Estimation for the user stories – Hospital Appointment Management System.

Poker Estimation

Points	Meaning
1	Simple UI text update or small bug fix
2	Login/Register form validation
3	Basic database connection and API setup
5	Appointment booking module with frontend and backend
8	Dashboard with appointment history and management
13	Integration of Azure DevOps, CI/CD pipeline, and database
21	Complete Hospital Management System with all modules and deployment

Procedure:

- Assign points:
  - Add Category → 3
  - Delete Category → 2
  - Search → 1

Sprint	Total SP	Duration	Velocity	Focus
Sprint 1	32 SP	2 weeks	~13SP/dev/sprint	Login, Registration, Database Setup, Basic UI
Sprint 2	48 SP	2 weeks	~31 SP/dev/sprint	Azure Pipelines, Testing, CI/CD Integration

Result:

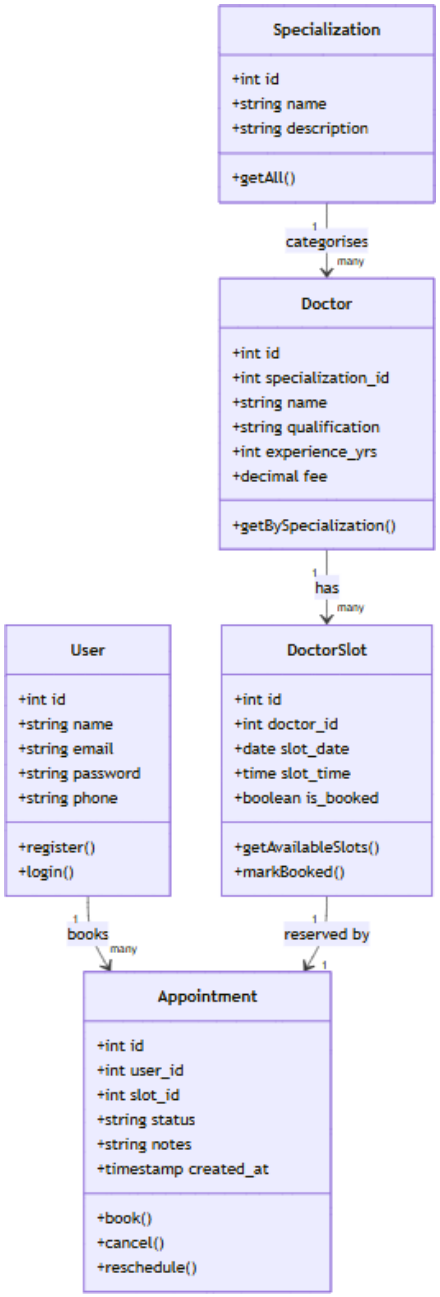
The Estimation/Story Points is created for the project using Poker Estimation.

EXP NO: 06	DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR PRODUCT CATEGORY MANAGEMENT SYSTEM

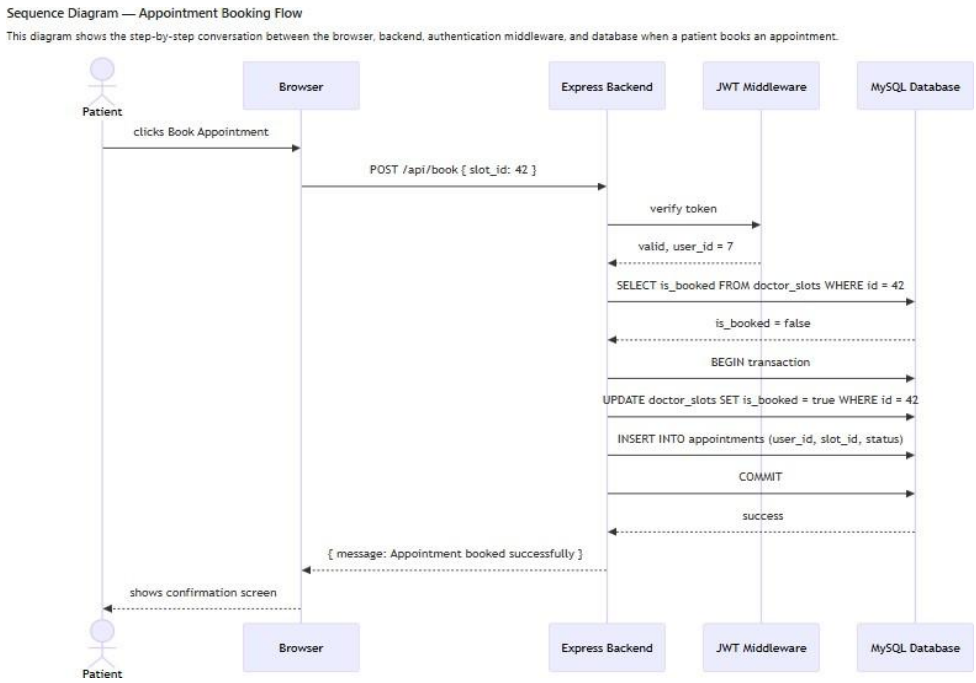
Aim:

To Design a Class Diagram and Sequence Diagram for the given Hospital Appointment Management System.

6A. Class Diagram



6B. Sequence Diagram



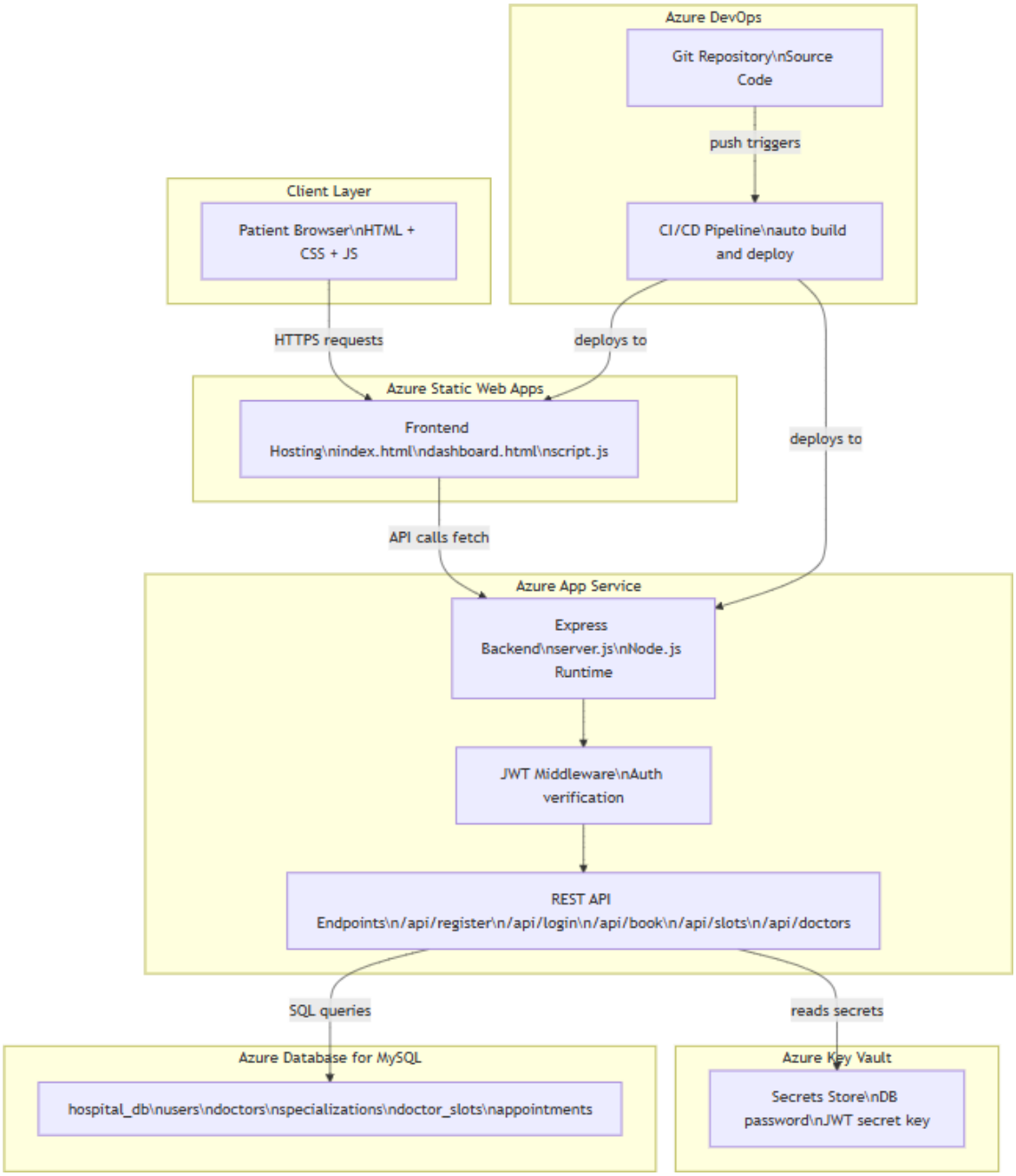
Result: The Class Diagram and Sequence Diagram is designed Successfully for the Hospital Appointment Management System

EXP NO: 07	DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR PRODUCT TRACKING SYSTEM

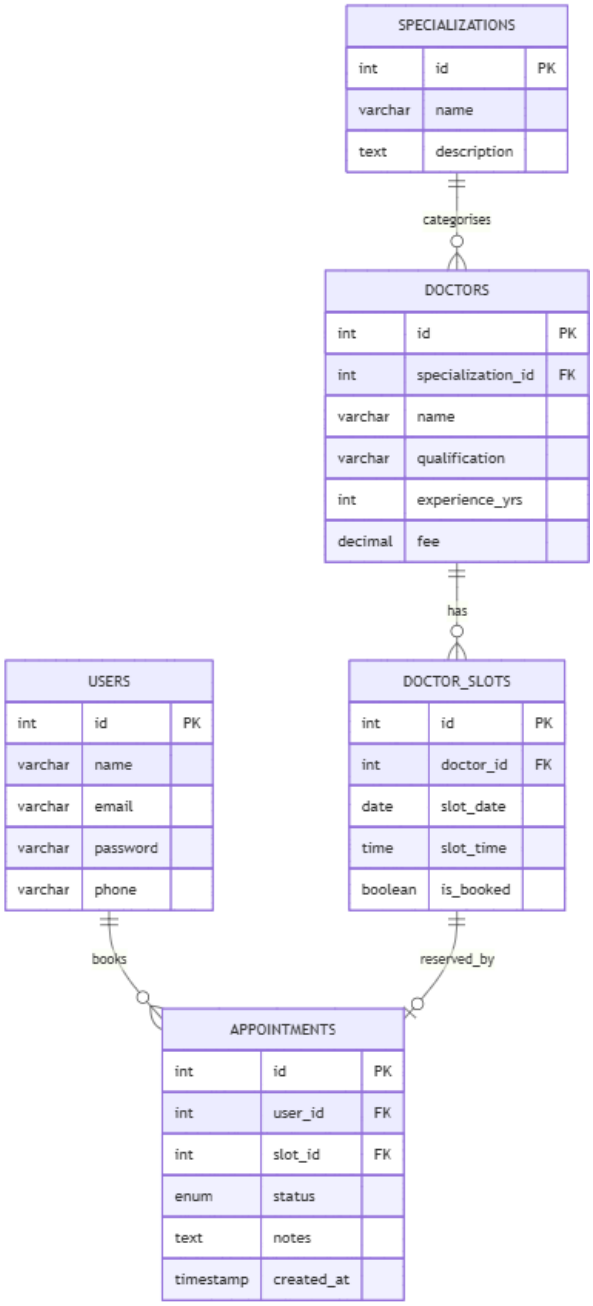
Aim:

To Design an Architectural Diagram and ER Diagram for the given Project.

7 A. Architectural Diagram:



7 B. ER Diagram



Result

The Architecture Diagram and ER Diagram is designed Successfully for the Hospital Appointment Management System.

EXP NO: 08	TESTING – TEST SUITES AND TEST CASES IN AZURE

Aim:
To create Test Plans and Test Cases for the Hospital Management System and validate core modules such as Login, Registration, Appointment Booking, Dashboard, and Appointment History using Azure DevOps Test Plans.

Test Planning and Test Case Design Procedure:

1. Understand Core Features of the Application
- User Registration
- User Login
- Dashboard Navigation
- Appointment Booking
- View Appointment History

2. Define User Interactions
- Each test case simulates real hospital management workflows.
- Both valid and invalid scenarios are tested.

3. Design Happy Path Test Cases
- Verify successful registration and login.
- Verify appointment booking flow.
- Verify dashboard accessibility.

4. Design Error Path Test Cases
- Invalid login credentials.
- Empty registration fields.
- Booking without selecting doctor/date.
- Invalid appointment inputs.

5. Break Down Steps and Expected Results
- Each test contains:
- User actions
- Input data
- Expected output
- Validation message

6. Use Clear Naming and IDs
- TC01 – Successful Login
- TC02 – Invalid Login
- TC03 – Successful Registration
- TC04 – Empty Registration
- TC05 – Appointment Booking

7. Separate Test Suites

- Login Management
- Registration Management
- Appointment Management
- Dashboard & History Validation

8. Prioritize and Review

- Core hospital workflows marked as High Priority.
- Test cases reviewed for completeness and traceability.

USER STORIES

- As a user, I want to register into the system securely so that I can access hospital services. (US01)
- As a user, I want to log in securely using my email and password. (US02)
- As a patient, I want to book appointments with doctors easily. (US03)
- As a user, I want to view my appointment details and booking history. (US04)
- As an admin/system, I want all appointment records stored in the database. (US05)
-

Test Suites

Test Suite: TS01 – User Registration
TC01 – Successful Registration
Action:

- Open Register page.
- Enter valid Name, Email, Password, and Phone Number.
- Click "Register".

Expected Results:

- Registration successful message displayed.
- User details stored in MySQL database.
- Redirected to Login page.

Type:
Happy Path

TC02 – Empty Registration Fields
Action:

- Open Register page.
- Leave fields empty.
- Click "Register".

Expected Results:

- Validation/error message displayed.
- Registration prevented.

Type:
Error Path

Test Suite: TS02 – User Login
TC03 – Successful Login
Action:

- Open Login page.
- Enter valid Email and Password.
- Click “Login”.

Expected Results:

- Login successful message displayed.
- Redirected to Dashboard page.

Type:
Happy Path

TC04 – Invalid Login
Action:

- Enter invalid email/password.
- Click “Login”.

Expected Results:

- “Invalid Email or Password” message displayed.

Type:
Error Path

Test Suite: TS03 – Appointment Booking
TC05 – Successful Appointment Booking
Action:

- Login successfully.
- Open Booking page.
- Select doctor and date.
- Click “Book”.

Expected Results:

- Appointment booked successfully.
- Data inserted into appointments table.
- Redirected to View page.

Type:
Happy Path

TC06 – Booking Without Doctor Selection
Action:

- Open Booking page.
- Leave doctor unselected.
- Click “Book”.

Expected Results:

- Validation message displayed.
- Booking prevented.

Type:
Error Path

Test Suite: TS04 – Dashboard and Appointment History
TC07 – Dashboard Access
Action:

- Login successfully.
- Navigate to Dashboard.

Expected Results:

- Dashboard displayed successfully.
- User options visible.

Type:
Happy Path

TC08 – View Appointment History
Action:

- Open View Appointments page.

Expected Results:

- Appointment records fetched from database.
- Appointment details displayed correctly.

Type:
Happy Path

TEST CASES:

70	TC07 – View Appointments	prajeet vijay	Design	hospital-appointment-system	4/23/2026 9:04:47 AM
69	TC06 – Booking (Empty Fields)	prajeet vijay	Design	hospital-appointment-system	4/23/2026 9:03:29 AM
68	TC05 – Appointment Booking (Valid)	prajeet vijay	Design	hospital-appointment-system	4/23/2026 9:02:06 AM
67	TC04 – Login (Invalid Password)	prajeet vijay	Design	hospital-appointment-system	4/23/2026 8:59:29 AM
66	TC03 – Login (Valid Credentials)	prajeet vijay	Design	hospital-appointment-system	4/23/2026 8:55:15 AM
65	TC02 – Registration (Duplicate Email)	prajeet vijay	Design	hospital-appointment-system	4/23/2026 8:53:45 AM
64	TC01 - Verify User Registration with valid data	prajeet vijay	Design	hospital-appointment-system	4/23/2026 8:48:39 AM

RESULT:

Successfully created and executed Test Plans and Test Cases for the Hospital Management System in Azure DevOps. Both happy path and error path scenarios were validated for Registration, Login, Appointment Booking, Dashboard, and Appointment History modules

EXP NO: 09	LOAD TESTING AND CI/CD PIPELINE SETUP

AIM:
Load Testing is used to simulate multiple users accessing the Hospital Management System simultaneously to evaluate application performance, response time, stability, and scalability under heavy traffic conditions. Azure Load Testing was used to monitor the application's behavior and identify performance bottlenecks before deployment.

Steps to Perform Load Testing Using Azure
1. Open Azure Portal
   - Sign in to Microsoft Azure Portal.
   - Open Azure Load Testing service.

2. Create Load Testing Resource
   - Click:

Create Resource
   - Enter:
     - Resource Name
     - Subscription
     - Resource Group
     - Region

3. Create a New Test
   - Open Azure Load Testing dashboard.
   - Click:

Create Test

4. Configure Test Settings

Parameter	Value
Test Type	URL Based Test
Users	50 Virtual Users
Duration	2 Minutes
Ramp-Up Time 30 Seconds	

4. Configure Endpoint

Field	Value
Server URL	http://localhost:3000
API Path	/login
Method	POST

6. Add Request Body

```
{
  "email": "test@mail.com",
  "password": "123456"
}
```

7. Run Load Test

- Save the configuration.
- Click:

Run Test

8. Monitor Performance Metrics

Observe:

- Response Time
- Requests per Second
- Throughput
- Error Percentage
- CPU and Memory Usage

Result:

Successfully performed Load Testing on the Hospital Management System using Azure Load Testing. Multiple virtual users accessed the Login API simultaneously, and system performance metrics such as response time, throughput, and server stability were analyzed successfully.

CI/CD Pipeline

Steps to Create and Demonstrate CI/CD Pipeline

1. Open GitHub Repository

Open the Hospital Management System repository in GitHub.

Repository:

Hospital-Management-System_Repo

2. Open GitHub Actions

- Navigate to:

Actions Tab

- Create a new workflow.

3. Create Workflow File
Create:
.github/workflows/nodejs.yml

4. Configure Pipeline Stages
The pipeline performs:

- Install dependencies
- Build Node.js application
- Run test cases
- Validate project files
- Trigger deployment workflow

5. Save Workflow File
Commit and push workflow configuration to GitHub.

6. Trigger Pipeline

- Push code changes to repository.
- GitHub Actions automatically starts CI/CD workflow.

7. Verify Pipeline Execution
Check:

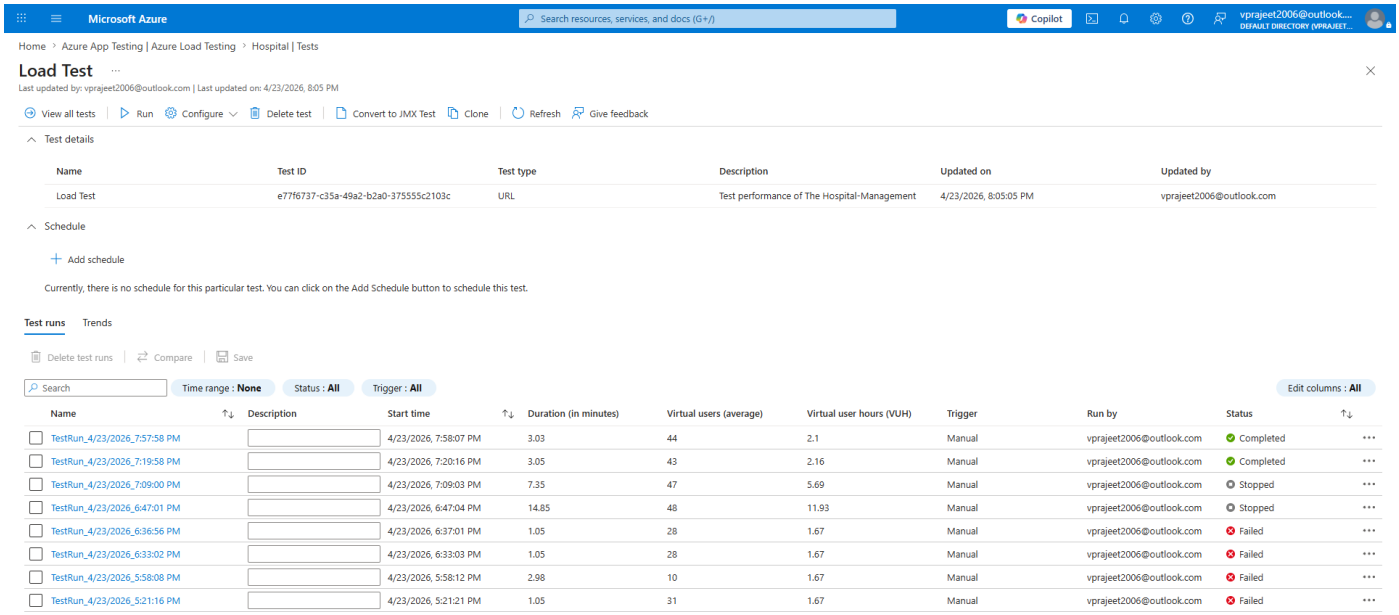
- Build Status
- Test Execution
- Deployment Logs
- Workflow Success Status

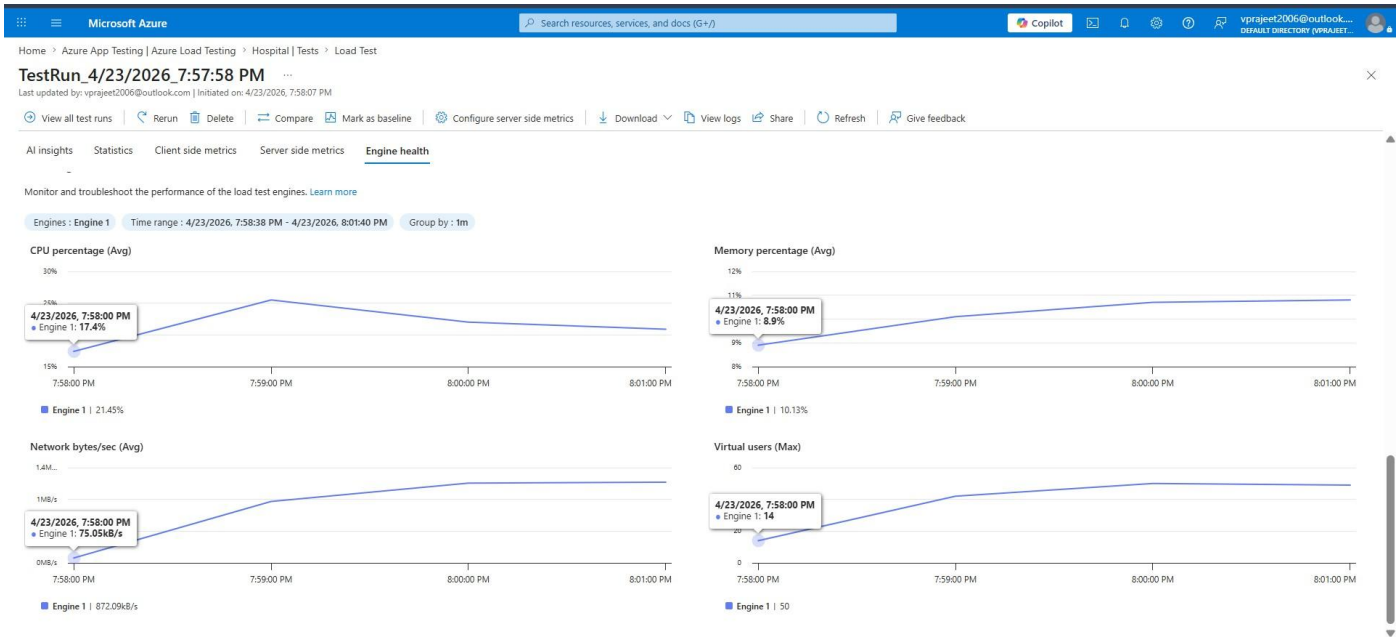
CI/CD Workflow Used
YAML

```yaml
name: Hospital Management CI/CD

on:
  push:
    branches:
      - main

jobs:
  build:

    runs-on: ubuntu-latest steps:

      - name: Checkout Code uses:
        actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: 18

      - name: Install Dependencies
        run: npm install
```

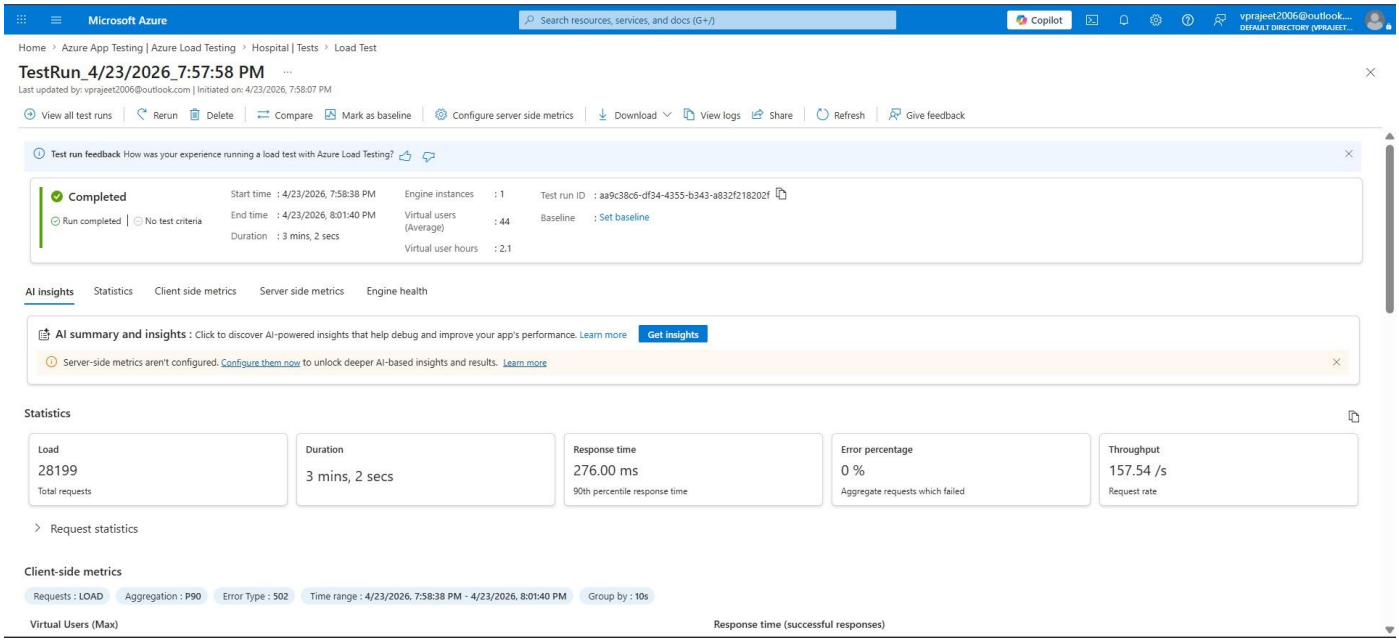
- name: Run Tests
  run: echo "Tests Executed Successfully"

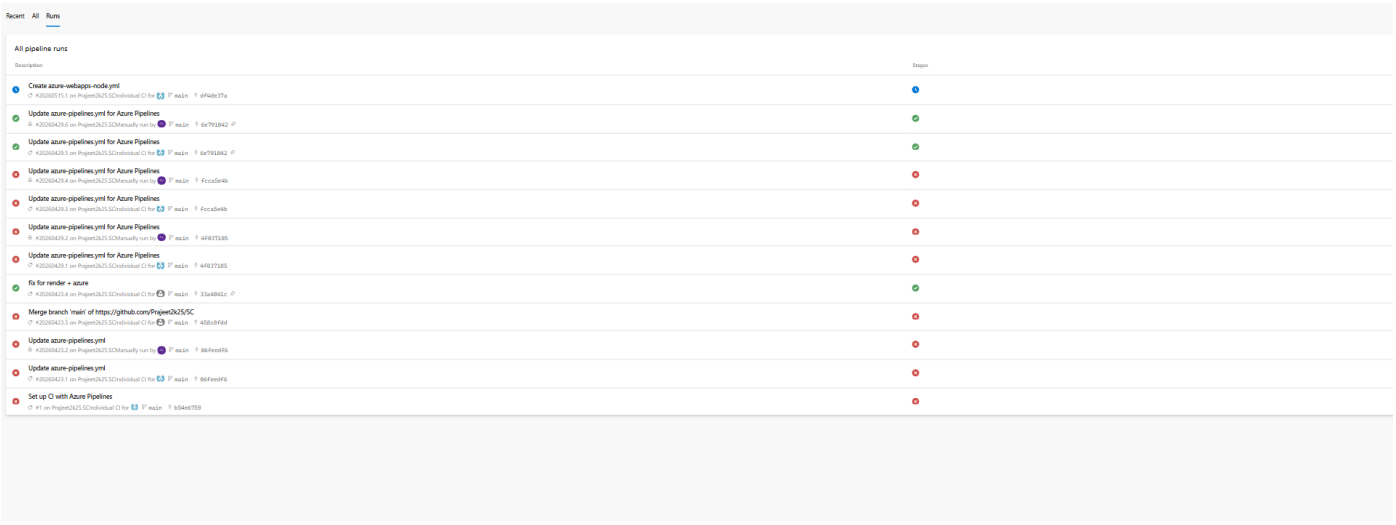
- name: Build Application run:
  echo "Build Successful"





Client-side metrics
Requests : LOAD
Aggregation : P90
Error Type : 502
Time range : 4/23/2026, 7:58:38 PM - 4/23/2026, 8:01:40 PM
Group by : 10s
Virtual Users (Max)
LOAD | 50
Response time (successful responses)
LOAD P90 | 1.01s
Requests/sec (Avg)
LOAD | 156.66
Errors (total)
LOAD 502 | 1


Microsoft Azure
Search resources, services, and docs (G+/)
Copilot
vprajeet2006@outlook....
DEFAULT DIRECTORY (VPRAJEET...
Home > Azure App Testing | Azure Load Testing > Hospital | Tests > Load Test
TestRun_4/23/2026_7:57:58 PM
Last updated by: vprajeet2006@outlook.com | Initiated on: 4/23/2026, 7:58:07 PM
View all test runs
Rerun
Delete
Compare
Mark as baseline
Configure server side metrics
Download
View logs
Share
Refresh
Give feedback
Test run feedback How was your experience running a load test with Azure Load Testing?
Completed
Run completed
No test criteria
Start time : 4/23/2026, 7:58:38 PM
End time : 4/23/2026, 8:01:40 PM
Duration : 3 mins, 2 secs
Engine instances : 1
Virtual users (Average) : 44
Virtual user hours : 2.1
Test run ID : aa9c38c6-df34-4355-b343-a832f218202f
Baseline : Set baseline
AI insights
Statistics
Client side metrics
Server side metrics
Engine health
AI summary and insights : Click to discover AI-powered insights that help debug and improve your app's performance. Learn more
Get insights
Server-side metrics aren't configured. Configure them now to unlock deeper AI-based insights and results. Learn more
Statistics
Load
28199
Total requests
Duration
3 mins, 2 secs
Response time
276.00 ms
90th percentile response time
Error percentage
0 %
Aggregate requests which failed
Throughput
157.54 /s
Request rate
Request statistics
Client-side metrics
Requests : LOAD
Aggregation : P90
Error Type : 502
Time range : 4/23/2026, 7:58:38 PM - 4/23/2026, 8:01:40 PM
Group by : 10s
Virtual Users (Max)
Response time (successful responses)


Recent
All
Runs
All pipeline runs
Description
Stages
Create azure-webapps-node.yml
Update azure-pipelines.yml for Azure Pipelines
Update azure-pipelines.yml for Azure Pipelines
Update azure-pipelines.yml for Azure Pipelines
Update azure-pipelines.yml for Azure Pipelines
Update azure-pipelines.yml for Azure Pipelines
Update azure-pipelines.yml for Azure Pipelines
Update azure-pipelines.yml for Azure Pipelines
fix for render + azure
Merge branch 'main' of https://github.com/Prajeet2k25/SC
Update azure-pipelines.yml
Update azure-pipelines.yml
Update azure-pipelines.yml
Set up CI with Azure Pipelines

l-appointment-system / Pipelines / Prajeet2k25 SC / 20260429.6

#20260429.6 • Update azure-pipelines.yml for Azure Pipelines

Prajeet2k25 SC

This run is being retained as one of 3 recent runs by pipeline.

Summary Code Coverage

Manually run by prajeet vijay

Apr 29 at 10:16 PM 47s

Repository and version
Prajeet2k25 SC
main 6e791042

Related
0 work items
0 published

Tests and coverage
Get started

Warnings 2

Task 'Node.js tool installer' version 0 (NodeTool@0) is deprecated.
Initialize job

This task is deprecated and will no longer receive updates. Please use UseNodeV1 as a replacement.
Initialize job

Stages Jobs

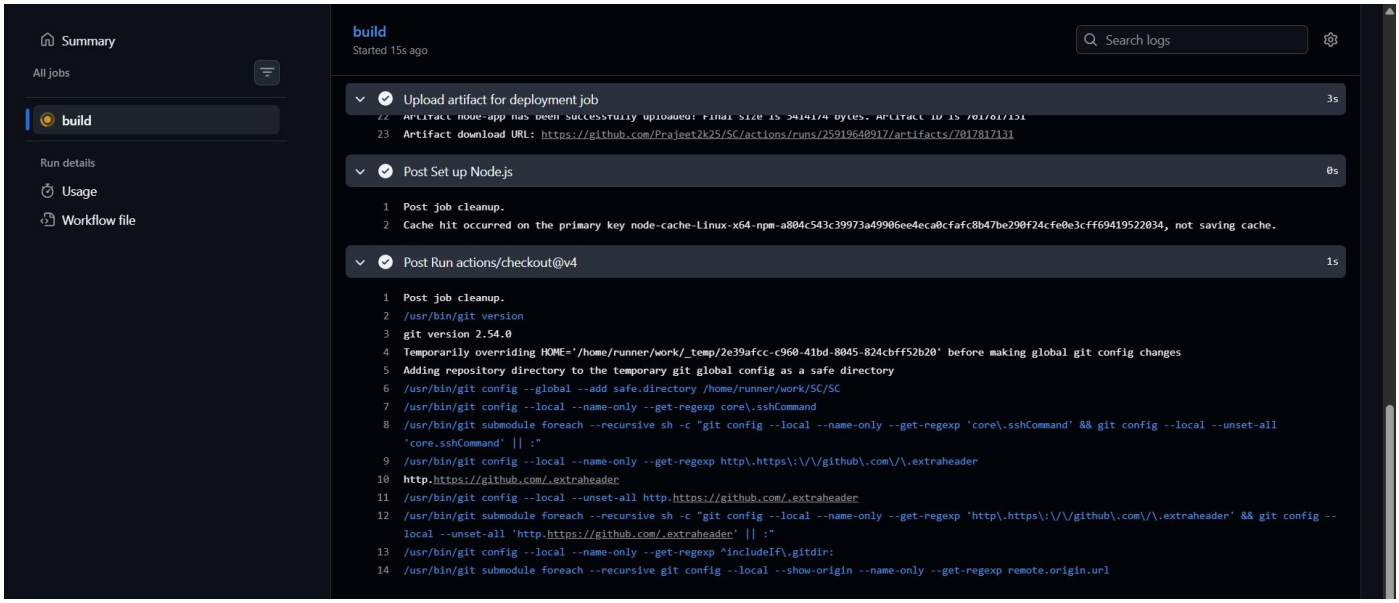
Stage
1 job completed 43s

Job 43s

Rerun Stage

Initialize job

```
Starting: Initialize job
Agent name: 'LATITUDE5520'
Agent machine name: 'LATITUDE5520'
Current agent version: '4.272.0'
Agent running as: 'User'
Prepare build directory.
Set build variables.
Download all required tasks.
##[warning]Task 'Node.js tool installer' version 0 (NodeTool@0) is deprecated.
##[warning]This task is deprecated and will no longer receive updates. Please use UseNodeV1 as a replacement.
Checking job knob settings.
   Knob: DockerActionRetries = true Source: $(VSTSAGENT_DOCKER_ACTION_RETRIES)
   Knob: UseGitLongPaths = true Source: $(USE_GIT_LONG_PATHS)
   Knob: UseNode24withHandlerData = True Source: $(DistributedTask.Agent.UseNode24withHandlerData)
   Knob: EnableIssueSourceValidation = true Source: $(ENABLE_ISSUE_SOURCE_VALIDATION)
   Knob: AgentEnablePipelineArtifactLargeChunkSize = true Source: $(AGENT_ENABLE_PIPELINEARTIFACT_LARGE_CHUNK_SIZE)
   Knob: ContinueAfterCancelProcessTreeKillAttempt = true Source: $(VSTSAGENT_CONTINUE_AFTER_CANCEL_PROCESSTREEKILL_ATTEMPT)
   Knob: ProcessHandlerSecureArguments = false Source: $(AZP_75787_ENABLE_NEW_LOGIC)
   Knob: ProcessHandlerSecureArguments = false Source: $(AZP_75787_ENABLE_NEW_LOGIC_LOG)
   Knob: ProcessHandlerTelemetry = true Source: $(AZP_75787_ENABLE_COLLECT)
   Knob: UseNewNodeHandlerTelemetry = True Source: $(DistributedTask.Agent.USENEWNODEHANDLERTELEMETRY)
   Knob: ProcessHandlerEnableNewLogic = true Source: $(AZP_75787_ENABLE_NEW_PH_LOGIC)
   Knob: EnableResourceMonitorDebugOutput = true Source: $(AZP_ENABLE_RESOURCE_MONITOR_DEBUG_OUTPUT)
   Knob: EnableResourceUtilizationWarnings = true Source: $(AZP_ENABLE_RESOURCE_UTILIZATION_WARNINGS)
   Knob: IgnoreVSTSTaskLib = true Source: $(AZP_AGENT_IGNORE_VSTSTASKLIB)
   Knob: FailJobWhenAgentDies = true Source: $(FAIL_JOB_WHEN_AGENT_DIES)
   Knob: EnhancedWorkerCrashHandling = true Source: ${AZP_ENHANCED_WORKER_CRASH_HANDLING}
   Knob: CheckForTaskDeprecation = true Source: $(AZP_AGENT_CHECK_FOR_TASK_DEPRECATION)
   Knob: CheckIfTaskNodeRunnerIsDeprecated246 = True Source: $(DistributedTask.Agent.CheckIfTaskNodeRunnerIsDeprecated246)
   Knob: EnableTimeoutLogFlushing = True Source: $(DistributedTask.Agent.EnableTimeoutLogFlushing)
   Knob: LogTaskNameInUserAgent = true Source: $(AZP_AGENT_LOG_TASKNAME_IN_USERAGENT)
   Knob: UseFetchFilterInCheckoutTask = true Source: $(AGENT_USE_FETCH_FILTER_IN_CHECKOUT_TASK)
   Knob: Rosetta2Warning = true Source: $(ROSETTA2_WARNING)
   Knob: CheckPsModulesLocations = False Source: $(DistributedTask.Agent.CheckPsModulesLocations)
   Knob: AddForceCredentialsToGitCheckoutEnhanced = True Source: $(DistributedTask.Agent.AddForceCredentialsToGitCheckoutEnhanced)
   Knob: UseLatestTfExe = True Source: $(DistributedTask.Agent.UseLatestTfExe)
   Knob: UseSparseCheckoutInCheckoutTask = true Source: $(AGENT_USE_SPARSE_CHECKOUT_IN_CHECKOUT_TASK)
   Knob: AgentCDNConnectivityFailWarning = False Source: $(DistributedTask.Agent.AgentCDNConnectivityFailWarning)
   Knob: UseEnhancedLogging = true Source: ${AZP_USE_ENHANCED_LOGGING}
Finished checking job knob settings.
Start tracking orphan processes.
Finishing: Initialize job
```



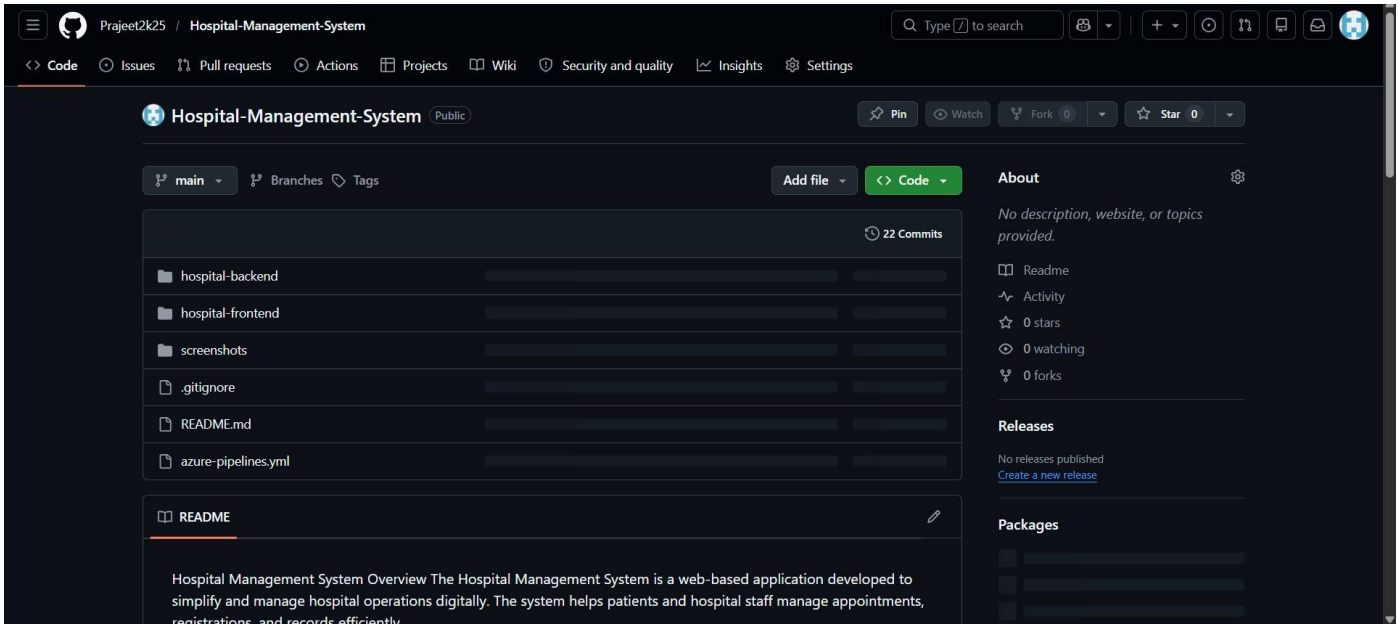
Result:
Successfully implemented and demonstrated a CI/CD pipeline for the Hospital Management System using GitHub Actions. The workflow automatically builds and validates the application whenever code changes are pushed to the reposit

EXP NO: 10	GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS

Aim:

To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the Product Tracking System project.

GitHub Project Structure



Result:

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the code.